

TOPS Technologies - SQL & PL/SQL Assessment

Section A: Concept Application

1. Relational databases maintain accuracy by using structured tables with relationships (primary and foreign keys), ensuring valid and consistent data.
2. Constraints ensure data integrity by preventing invalid entries like duplicate IDs or null values.
3. GROUP BY helps analyze spending by grouping data and applying aggregate functions like SUM.
4. Rollback is used when incorrect expense data is inserted, allowing undo of changes.
5. Views simplify queries and help users track monthly expenses efficiently.
6. Triggers automate updates such as balance calculation after inserting expenses.

Section B: SQL Hands-On

Insert Users

```
INSERT INTO users VALUES
(1, 'Parth', 'parth@gmail.com', '2026-01-01'),
(2, 'Rahul', 'rahul@gmail.com', '2026-01-02'),
(3, 'Amit', 'amit@gmail.com', '2026-01-03'),
(4, 'Neha', 'neha@gmail.com', '2026-01-04'),
(5, 'Priya', 'priya@gmail.com', '2026-01-05');
```

Insert Categories

```
INSERT INTO categories VALUES
(1, 'Food'),
(2, 'Rent'),
(3, 'Entertainment');
```

Insert Expenses

```
INSERT INTO expenses VALUES
(1,1,1,200.50,'2026-02-01'),
(2,1,2,5000.00,'2026-02-02'),
(3,2,1,150.00,'2026-02-03'),
(4,2,3,800.00,'2026-02-04'),
(5,3,2,6000.00,'2026-02-05'),
(6,3,1,300.00,'2026-02-06'),
(7,4,3,1200.00,'2026-02-07'),
(8,4,1,250.00,'2026-02-08'),
(9,5,2,7000.00,'2026-02-09'),
(10,5,3,900.00,'2026-02-10');
```

Update Query

```
UPDATE expenses SET amount = 1000 WHERE expense_id = 4;
```

Delete Query

```
DELETE FROM expenses WHERE amount < 200;
```

Join Query

```
SELECT e.expense_date, e.amount, u.name, c.category_name
FROM expenses e
JOIN users u ON e.user_id = u.user_id
JOIN categories c ON e.category_id = c.category_id;
```

Group By

```
SELECT c.category_name, SUM(e.amount)
FROM expenses e
JOIN categories c ON e.category_id = c.category_id
GROUP BY c.category_name;
```

Sort Users by Spending

```
SELECT u.name, SUM(e.amount) AS total
FROM users u
JOIN expenses e ON u.user_id = e.user_id
GROUP BY u.name
ORDER BY total DESC;
```

View Creation

```
CREATE VIEW ActiveUsersView AS
SELECT u.name, u.email
FROM users u
JOIN expenses e ON u.user_id = e.user_id
GROUP BY u.user_id
HAVING COUNT(e.expense_id) > 5;
```

View Query

```
SELECT * FROM ActiveUsersView;
```

Section C: Mini Project

CRUD

```
INSERT INTO expenses VALUES (11,1,1,400,'2026-02-11');
SELECT * FROM expenses;
```

```
UPDATE expenses SET amount=450 WHERE expense_id=11;  
DELETE FROM expenses WHERE expense_id=11;
```

Stored Procedure

```
DELIMITER $$  
CREATE PROCEDURE GetMonthlyExpense(IN uid INT)  
BEGIN  
SELECT SUM(amount) FROM expenses WHERE user_id = uid;  
END$$  
DELIMITER ;
```

Transaction

```
START TRANSACTION;  
INSERT INTO expenses VALUES (12,1,1,9999,'2026-02-12');  
ROLLBACK;  
COMMIT;
```